

TP : MÉTHODE D'EULER, DIFFÉRENCES FINIES

Objectifs du TP

- Réviser la méthode d'Euler.
- Tracer des courbes.
- Discrétiser des dérivées d'ordre supérieur.
- Résoudre numériquement des problèmes aux limites.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint
```

1. Révisions sur la méthode d'Euler (1 heure)

On souhaite résoudre numériquement l'équation différentielle suivante (modélisation d'une chute libre d'un parachutiste avant qu'il n'ouvre son parachute) :

$$\begin{cases} \ddot{z} &= \frac{C}{m} e^{-\frac{z}{h}} \dot{z}^2 - g \\ z(t_0) &= z_0 \\ \dot{z}(t_0) &= 0 \end{cases}$$

1. Prendre un papier et un crayon et retrouver le schéma d'Euler¹ explicite pour une équation différentielle $y'(t) = F(y(t), t)$.

2. Écrire une fonction `Euler(F, Y0, T)` prenant en argument

- Une fonction F permettant de décrire l'équation différentielle $y' = F(y, t)$,
- Une condition initiale $Y_0 = y(t_0)$,
- Un tableau ou une liste de temps T dont le premier élément est l'instant initial t_0 ,

et renvoyant un tableau Numpy de format (n, d) où n est la taille de T , d la dimension de Y_0 , et contenant les valeurs approchées des $y(T_k)$.

⚠ Y_0 pourra être au départ un tableau Numpy (cas d'une équation en dimension > 1) : tirer parti de la vectorisation avec Numpy. On peut englober le cas où Y_0 est scalaire en débutant la fonction par `y0 = np.array(Y0, ndmin=1)` afin de le convertir en array.

3. Transformer l'équation différentielle du parachutiste et une équation d'ordre 1 en $\begin{pmatrix} z \\ \dot{z} \end{pmatrix}$, et donner la fonction F correspondante.

1.



Leonhard Euler (Bâle, 1707 - Saint-Petersbourg 1783) est un scientifique suisse à l'œuvre considérable en astronomie, en sciences physiques et en mathématiques. Il est entre autre à l'origine du concept de fonction qui permis à l'analyse de se développer, de la formule qui porte son nom reliant le nombre de S sommet, A d'arrêtes et F de faces d'un polyèdre convexe : $F - A + S = 2$. Il fit d'importantes découvertes en calcul différentiel et intégral, géométrie et théorie des nombre. On lui doit par exemple la formule $\sum_{k \geq 1} \frac{1}{k^2} = \frac{\pi^2}{6}$ ou la relation $e^{i\theta} = \cos\theta + i\sin\theta$. C'est d'ailleurs lui qui introduisit le symbole Σ pour les sommations. Il fût aveugle pendant 12 ans, ce qui ne l'empêcha pas d'être extraordinairement prolifique avec 886 ouvrages réunis en 80 volumes.

4. On va appliquer la résolution numérique au saut record(s) de Félix Baumgartner effectué le 14 octobre 2012², premier homme à dépasser le mur du son en chute libre.

On donne $C = 0,20 \text{ kg} \cdot \text{m}^{-1}$ et $h = 8,0 \text{ km}$.

On suppose que sa masse est $m = 90 \text{ kg}$, et on suppose $g = 9,8 \text{ m} \cdot \text{s}^{-2}$ constant à toute altitude.



D'après Wikipedia :

L'altitude de départ est 39,376 km, la chute libre a duré 4 minutes et 19 secondes, et il a atteint la vitesse maximale de $1342,8 \text{ km} \cdot \text{h}^{-1}$ – au bout de $45,5 \pm 0,5$ secondes – avant d'ouvrir son parachute à 2 500 m d'altitude (et une vitesse d'équilibre alors d'environ $200 \text{ km} \cdot \text{h}^{-1}$) et de se poser sans encombre après une chute totale de 9 min 3 s.

Remarque : La fonction `Euler` renvoie un tableau Numpy dont les colonnes contiennent les valeurs de z et de $\dot{z}$.

Si $Y = \text{Euler}(F, T, [z0, dz0])$, il suffit d'écrire $Z, dZ = Y.T$ (transposition) pour récupérer les valeurs de chacun, ce qui, avec le tableau T (obtenu par `linspace` ou `arange`) permet de tracer facilement le résultat.

On peut aussi écrire $Z = Y[:, 0]$ et $dZ = Y[:, 1]$.

Résoudre numériquement l'équation différentielle, tracer successivement l'altitude en fonction de t , la vitesse en fonction de t et le portrait de phase. Retrouver les observations ci-dessus.

5. Consulter `help(odeint)` et retrouver les résultats en utilisant la fonction `odeint` (qui fonctionne de manière similaire à votre fonction `Euler`).

2. La méthode des différences finies pour un problème aux limites en dimension 1³ (1 heure)

A. Présentation du problème

On souhaite résoudre numériquement un problème du type

$$\begin{cases} \forall x \in [0, 1], & -u''(x) + c(x)u(x) = f(x) \\ u(0) = \alpha, & u(1) = \beta \end{cases}$$

d'inconnue $u \in \mathcal{C}^2([0, 1])$, et où les deux fonctions $c, f \in \mathcal{C}^0([0, 1])$ et les deux scalaires $\alpha, \beta \in \mathbb{R}$ sont fixés.

Un tel problème est appelé **problème aux limites**, car la fonction inconnue doit satisfaire des **conditions aux limites** $u(0) = \alpha$ et $u(1) = \beta$.

Contrairement au problème de Cauchy (avec des conditions initiales), on n'a pas de garantie en général d'existence et d'unicité des solutions aux problèmes aux limites. On peut montrer qu'une condition suffisante (mais non nécessaire) est d'avoir $c \geq 0$, et on n'a pas de méthode général pour résoudre de manière exacte ces problèmes.

2. voir https://fr.wikipedia.org/wiki/Felix_Baumgartner

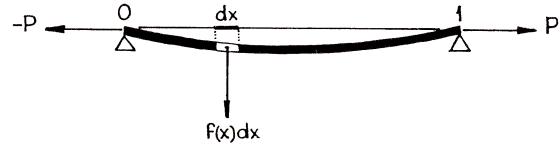
3. Pour prolonger cette étude, on pourra consulter l'excellent *Introduction à l'analyse numérique matricielle et à l'optimisation* de Philippe Ciarlet.

Le livre de Ciarlet nous explique que :

Un exemple de situation physique où il est rencontré est celui du fléchissement d'une poutre de longueur 1, étirée selon son axe par une force P , soumise à une charge transversale $f(x)dx$ par unité de longueur dx et simplement appuyée à ses extrémités 0 et 1.

Alors le moment fléchissant $u(x)$ au point d'abscisse x est solution d'un problème aux limites du type ci-dessus, avec $c(x) = \frac{P}{E \cdot I(x)}$ où E est le module d'Young du ma-

tériau constituant la poutre et $I(x)$ le moment principal d'inertie de la section de la poutre au point d'abscisse x , et avec $\alpha = \beta = 0$.



B. Discrétisation par la méthode des différences finies

Pour approcher numériquement la solution d'un problème aux limites, il faut discrétiser la dérivée seconde.

Pour cela, on subdivise régulièrement le segment $[0, 1]$ avec $n + 2$ points $x_0 = 0, x_1, \dots, x_{n+1} = 1$ et donc un pas de $h = \frac{1}{n+1}$ (et pour tout $i \in \llbracket 0, n+1 \rrbracket$, $x_i = ih$.)

Si u est solution exacte du problème, on cherche à obtenir $(u_1, \dots, u_n)$ tel que pour tout $i \in \llbracket 1, n \rrbracket$, $u_i \approx u(x_i)$, et $u_0 = u(0) = \alpha$, $u_{n+1} = u(1) = \beta$.

1. Prendre un papier et un crayon (oui, oui) et démontrer rigoureusement que si u est de classe $\mathcal{C}^2$ sur $[0, 1]$ et $x \in [0, 1]$ fixé,

$$u''(x) = \frac{u(x-h) - 2u(x) + u(x+h)}{h^2} + o_{h \rightarrow 0}(1)$$

On propose donc de discrétiser l'équation différentielle en posant pour tout $i \in \llbracket 1, n \rrbracket$, $c_i = c(x_i)$, $f_i = f(x_i)$ et

$$-\frac{u_{i-1} - 2u_i + u_{i+1}}{h^2} + c_i u_i = f_i$$

avec toujours $u_0 = \alpha$ et $u_{n+1} = \beta$.

2. Montrer (toujours sur papier) que ce schéma se ramène à un système d'équation $Au = b$ où $u = \begin{pmatrix} u_1 \\ \vdots \\ u_n \end{pmatrix}$ avec A une matrice carrée plutôt simple et b un second membre à préciser.

Voyez-vous une propriété simple vérifiée par la matrice A qui la rend inversible si $c \geq 0$?

Il suffit donc de résoudre ce système linéaire pour obtenir une solution approchée. On peut montrer que si la solution exacte est de classe $\mathcal{C}^4$, l'erreur est, en norme infinie, en $O(h^2)$.

3. Écrire une fonction `matrice_A(h, tab_c)` prenant en argument le pas h et le tableau `tab_c` de $(c_1, \dots, c_n)$ (de taille n) et renvoyant le tableau représentant la matrice A (de taille $n \times n$).

On tirera avantage de la vectorisation et on pourra utiliser avantageusement la fonction `np.diag` (voir son aide).

On demande d'écrire cette fonction sans aucune boucle.

4. Écrire une fonction `differences_finies(c, f, n, alpha, beta)` prenant en argument les fonctions f et c et l'entier n , les conditions aux limites α et β et renvoyant les tableaux donnant $(x_0, \dots, x_{n+1})$ et $(u_0, \dots, u_{n+1})$ **en tirant parti au maximum des tableaux numpy**. On pourra vectoriser c et f au début de la fonction pour ne pas avoir de problème :

```
c = np.vectorize(c)
f = np.vectorize(f)
```

On demande d'écrire cette fonction sans aucune boucle.

La fonction `np.linalg.solve` permet de résoudre numériquement des systèmes linéaires.

5. Tracer dans une même fenêtre, la solution exacte et la solution approchée des problèmes suivants, avec $n = 100$ pour :

5.a) $u'' = 1$ et $u(0) = u(1) = 0$.

5.b) $-u''(x) + x \cdot u(x) = (1 + 2x - x^2)e^x$ et $u(0) = 1, u(1) = 0$; solution exacte : $u(x) = (1 - x)e^x$.

6. Pour les deux exemples, calculer l'erreur $\|u(x_i) - u_i\|_\infty$.

7. Pour le deuxième exemple, calculer, pour quelques valeurs de n , $\frac{\|u(x_i) - u_i\|_\infty}{h^2}$, c'est-à-dire $\|u(x_i) - u_i\|_\infty \cdot (n+1)^2$. Que remarque-t-on ?

Remarque : On peut facilement étendre cette méthode aux dimensions supérieures pour résoudre des problèmes de Laplace (ou de Poisson) aux limites de la forme $\Delta u(x) = f(x)$ sur un domaine et $u(x) = g(x)$ sur le bord du domaine.